



game.py
Python



README.md
File

I have cloned this Python Pygame Dino Run project. Read the README and inspect the entire code carefully. First identify the deliberate high-score bug and explain exactly why it is wrong. Then fix only that bug without changing unrelated behaviour. Show me the changes clearly.

Worked for 13s >

I inspected the full `README.md` and all 195 lines of `game.py`. The README explicitly says there is **one deliberate bug**, and it points to the comparison immediately after a collision; the three empty functions are optional features, not part of the bug. README

The deliberate high-score bug

Right after a collision, the game calculates the score for the run:

Python

Run

```
current = self.score // 10
```

and then currently does:

Python

Run

```
if self.high_score > current:
    self.high_score = current
    save_high_score(self.high_score)
```

The problem is that the comparison is **backwards**. `self.high_score` represents the best score from previous runs, while `current` represents the run that just ended. The update should happen when:

```
current > self.high_score
```

The existing condition does the exact opposite.

For example, suppose:

```
Previous best = 20
Current run   = 35
```

The code checks:

Python

Run

```
20 > 35 # False
```

So the new best score is **never saved**.

Worse, if:

```
Previous best = 35
Current run   = 20
```

then:

Python

Run

```
35 > 20 # True
```

and the program actually replaces the best score with the **lower** score of 20.

That is exactly why the README says the comparison is backwards. README The surrounding implementation confirms that `high_score` is loaded at game startup and that `save_high_score()` writes the value to `dino_highscore.txt`. game

The only change

Diff

```

    if hit:
        self.state = "lose"
        current = self.score // 10
-       if self.high_score > current:
+       if current > self.high_score:
            self.high_score = current
            save_high_score(self.high_score)
    return
```

This is in the collision/high-score section of `Game.update`. game

I **did not change** the scoring logic, collision behavior, spawning, jumping, speed ramp, restart behavior, persistence functions, or any of the three optional feature functions. The fixed file also passes a Python syntax check.

Download the fixed `game.py`

`</>` **game_fixed.py**
Code

Sources



Now implement Task 2 from the README: implement `dino_tint(on_ground)` so the dino uses its normal green colour on the ground and a different colour while airborne. Keep the rest of the game unchanged. Just give me the updated code to copy paste or you may give the whole file-- whichever takes less tokens

Just replace the empty `dino_tint()` function with this:

Python

Run

```
def dino_tint(on_ground):  
    if on_ground:  
        return None  
    return (255, 165, 0)
```

That works with the existing line:

Python

Run

```
color = dino_tint(self.on_ground) or self.color
```

So:

- **On ground:** returns `None` → uses the existing normal green `(83, 141, 78)`
- **Airborne:** returns `(255, 165, 0)` → dino becomes orange

This matches Task 2's requirement without changing anything else.

README

Sources



That worked. Now implement Task 3 from the README: implement `on_obstacle_passed(obstacle, score)`. Add a simple visible `+1` popup whenever an obstacle is successfully passed. Keep the implementation simple and compatible with the existing code. You may give me the file to download

Worked for 20s >

Done. Task 3 is implemented with a simple `+1` popup when an obstacle is successfully passed. The existing Task 2 orange airborne tint and the Task 1 high-score fix are also preserved; `max_jumps()` remains untouched.

[Download the updated `game\_task3.py`](#)

The Task 3 change is essentially:

Python

Run

```
def on_obstacle_passed(obstacle, score):  
    obstacle.popup_timer = 30
```

and the obstacle draws `+1` briefly above itself. No scoring, collision, spawning, or other game behavior was changed.

 **game_task3.py**
Code



Now implement Task 4 from the README: implement `max_jumps()` to allow a double jump. Do not break the existing jump bookkeeping or other game behaviour.

Replace the empty `max_jumps()` function with this:

Python

Run

```
def max_jumps():  
    return 2
```

That's all Task 4 requires. The existing bookkeeping already:

- increments `jump_count` on each jump,
- prevents a third jump because `max_jumps()` returns `2`,
- resets `jump_count` to `0` when the dino lands. `game +1`

This gives you a **double jump** without changing any other game behavior, exactly as the README specifies. `README`

Sources



create a readme file, describing the game, how it works, and then what change logs/fixes were made

Done. I created a README covering the game, how its systems work, controls, project structure, and the changes made across Tasks 1–4.

 [Download the updated README.md](#) `</>`



README_updated.md
Document



Is this conversation helpful so far?



ChatGPT can make mistakes. Check important info.